Kangaroo: Caching Billions Of Tiny Objects On Flash

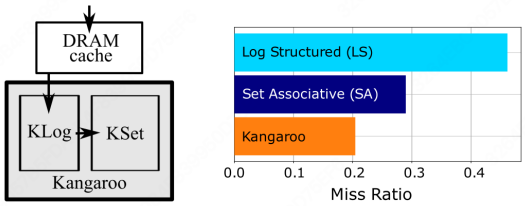
February 2, 2026

1 Introduction

Flash's main challenge is its limited write endurance; i.e., flash can only be written so many times before wearing out. Wearout is especially problematic for tiny objects because flash can be read and written only at multi-KB granularity. For example, writing a 100 B object may require writing a 4 KB flash page, amplifying bytes written by 40× and rapidly wearing out the flash device. Thus, cost also argues for minimizing excess bytes written to flash.

The Problem. Prior flash-cache designs either use too much DRAM or write flash too much.

- LSM: tracking billions of tiny objects requires a large index, and even a heavily optimized index needs large amounts of DRAM
- Set-associative caches operate by hashing objects' keys into distinct "sets," much like CPU caches. Admitting a single small object to the cache requires re-writing an entire set, significantly amplifying the number of bytes written to the flash device.



(a) Overview. ***(b)*** Kangaroo reduces misses by 29%.

Fig. 1. (a) High-level illustration of Kangaroo's design. (b) Miss ratio achieved on a production trace from Facebook by different flash-cache designs on a 1.9 TB drive with a budget of 16 GB DRAM and three device-writes per day. Prior designs are constrained by either DRAM or flash writes, whereas Kangaroo's design balances these constraints to reduce misses by 29%.

Figure 1

- To avoid a large DRAM index, Kangaroo organizes the bulk of cache capacity as a set-associative cache, called *KSet*.
- To reduce flash writes, Kangaroo places a small (e.g., 5% of flash) log-structured cache, called *KLog*, in front of KSet. KLog buffers many objects, looking for objects that map to the same set in KSet (i.e., hash collisions), so that each flash write to KSet can insert multiple objects.

Our insight is that even a small log will yield many hash collisions, so only a small amount of extra DRAM (for KLog's index) is needed to significantly reduce flash writes (in KSet).

2 Background and related work

2.1 Tiny objects are important and numerous

Tiny objects are prevalent in many large-scale systems:

- At Facebook, small objects are prevalent in the social graph. For example, the average social-graph edge size is under 100 B. Across edges, nodes, and other objects, the average object size is less than 700 B. This has led to the development of a dedicated flash caching system for small objects.
- At Twitter, tweets are limited to 280 B, and the average tweet is less than 33 characters. Due to the massive and growing number of tweets, Twitter seeks a cost-effective caching solution.

- At Microsoft Azure, a growing use case is processing updates from sensor data, such as from IoT devices in Azure Streaming Analytics. Before an update can be processed (e.g., to trigger a real-time action), the server must fetch metadata (the sensor's unit of measurement, geolocation, owner, etc.) with an average size of 300 B. For efficiency and availability, it caches the most popular metadata. Another use case arises in search advertising, where Azure caches predictions and other results

2.2 Caching tiny objects in flash is hard

DRAM is expensive

Challenges:

- flash has *limited write endurance*, which means there is a limit on the number of writes before the flash device wears out and must be replaced
- *write amplification*: Flash devices suffer from both device-level write amplification and application-level write amplification

Device-level write amplification (dlwa) occurs when the flash translation layer (FTL) writes more flash pages than asked for by the storage application (e.g., file system, database, or cache). Current flash drives implement the age-old block-storage interface, wherein hosts read and write logical blocks in a numerical logical-block address (LBA) namespace. Generally, the internal flash-page size and the external logical-block size are the same, with 4 KB being common, even though the flash device can only erase pages in much larger (e.g., 256 MB) "erase blocks". Most dlwa is caused by cleaning activity that copies live pages elsewhere before an erase block is cleared.

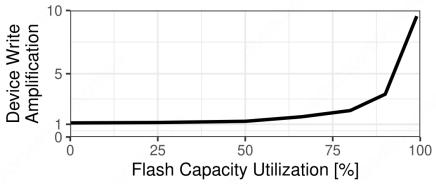


Fig. 2. The effect of flash over-provisioning on device-level write amplification (DLWA) of random writes of various sizes. DLWA increases as over-provisioning decreases.

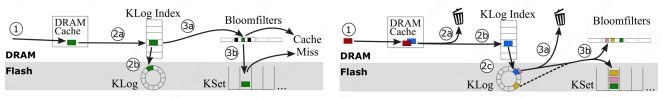
Figure 2

Generally speaking, dlwa worsens as more of the raw flash capacity is utilized and as access patterns consist more of small, random writes. A common approach to reduce dlwa is over-provisioning, i.e., only exposing a fraction of the raw flash capacity in the LBA namespace, so that cleaning tends to find fewer live pages in victim erase blocks. Fig. 2 shows dlwa vs. utilized capacity for random 4 KB writes to a 1.9 TB flash drive. As expected, dlwa significantly increases as over-provisioning decreases, from ≈1× at 50% utilization to ≈10× at 100% utilization.

Application-level write amplification (alwa) occurs when the storage application re-writes some of its own data as part of its storage management. One form of this is akin to FTL cleaning, such as cleaning in log-structured file systems or compaction in log-structured merge trees. Another form is caused by having to write an entire logical block. To write a smaller amount of data, the application must read the block, install the new data, and then write the entire block. For example, installing 1 KB of new data into a 4 KB logical block involves rewriting the other 3 KB, giving alwa of 4×. Ideally, the unmodified data in the block would not have been rewritten.

2.3 Current approaches and related work

3 Kangeroo Overview and Motivation



(a) A lookup in Kangaroo. ***(b)*** Insertion and eviction in Kangaroo.

Fig. 3. Overview of Kangaroo. Objects first go to a tiny DRAM cache; then KLog, a small on-flash log-structured cache with an in-DRAM index; and finally KSet, a large on-flash set-associative cache. KLog minimizes flash writes, and KSet minimizes DRAM usage.

Figure 3

Lookup:

1 lookups first check the DRAM cache, which is very small (<1% of capacity).

2 If the requested key is not found, requests next check KLog (≈5% of capacity). KLog maintains a DRAM index to track objects stored in a circular log on flash.

3 If the key is not found in KLog's index, requests check KSet (≈95% of capacity). KSet has no DRAM index; instead, Kangaroo hashes the requested key to find the set (i.e., the LBA(s) on flash) that might hold the object.

3a If the requested key is not in the small, per-set Bloom filter, the request is a miss.

3b Otherwise, the object is probably on flash, so the request reads the LBA(s) for the given set and scans for the requested key.

4 Problems

5 References